PES UNIVERSITY

Department of Computer Science and Engineering • Software Engineering & Testing Lab

Lab 7: Fuzzing for Input Validation Bugs

Student Name: Pranav K J

SRN: PES2UG24CS363

Assigned Repo: L6_L7_55_car_insurance_premium (Folder: LAB7)

GitHub Repository: github.com/Pranav-kj/L6_L7_55_car_insurance_premium

Environment: Python 3.11.9, pytest 8.3.4, hypothesis 6.170.15

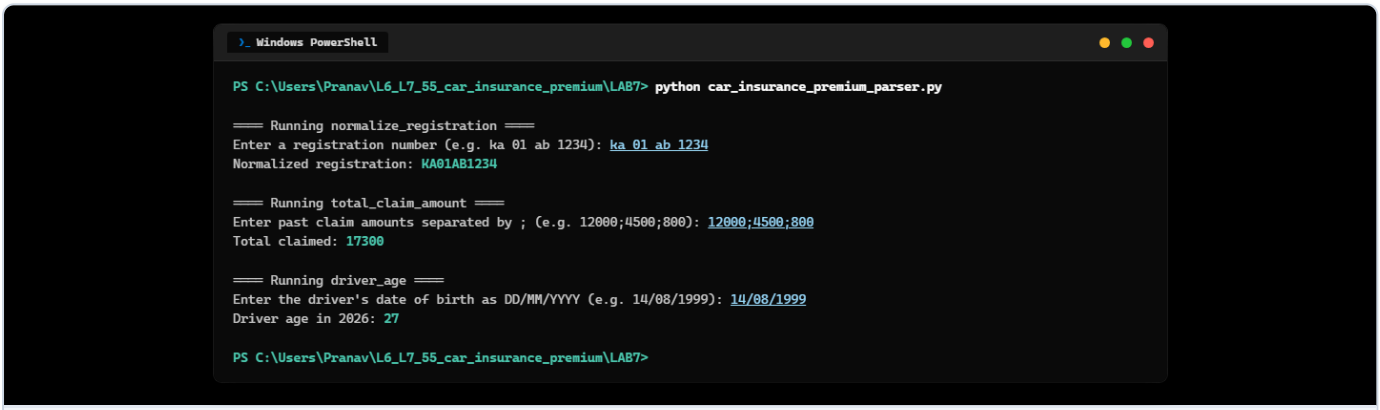
Submission Date: November 11, 2026

1. Objectives and Methodology

The objective of this lab is to explore **fuzz testing** as a security and quality assurance technique using Python's **Hypothesis** library. Property-based fuzzing tests invariants by automatically feeding hundreds of semi-random, malformed, and edge-case inputs to identify unexpected crashes and vulnerabilities. We then harden the assigned parsing module (`car_insurance_premium_parser.py`) using defensive programming.

2. Step 2: Running the Buggy Starter Application (SS1)

The starter script parses vehicle registration plates, past claim amounts, and driver dates of birth under the assumption that all input data is well-formed. When executed with standard happy-path inputs, the program functions normally without revealing underlying edge-case flaws.



Screenshot 1 (SS1): Output of `python car_insurance_premium_parser.py` on the starter module with standard valid inputs.

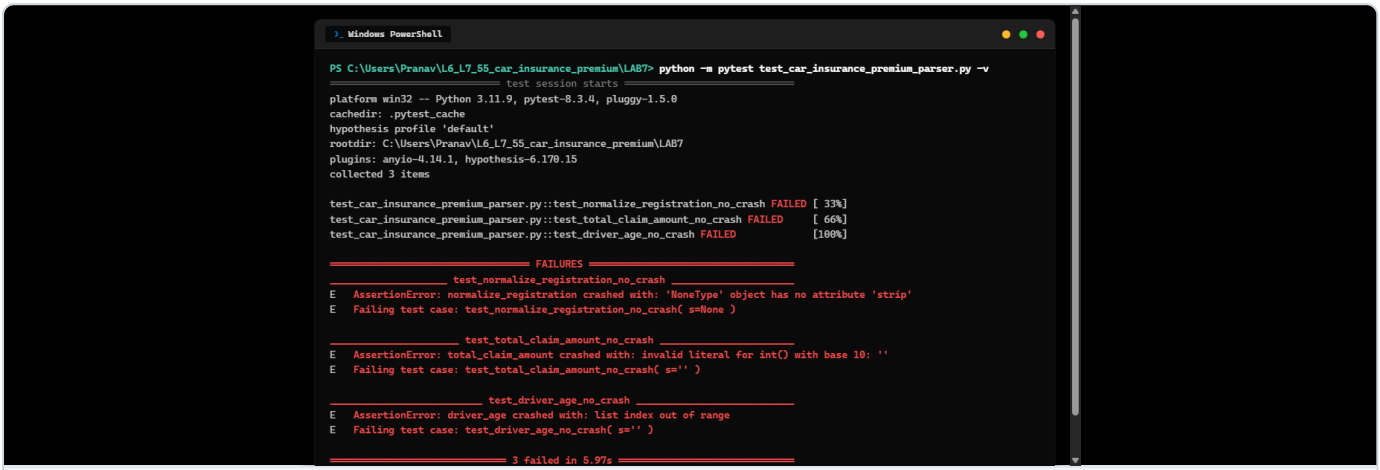
3. Step 3: Property-Based Fuzz Test Implementation

In `test_car_insurance_premium_parser.py`, we implemented Hypothesis test harnesses for each function using the strategy `@given(st.text() | st.none())`. The core property tested is that *no input—whether random Unicode characters, empty strings, delimiters, or None—should cause unhandled crashes*.

4. Step 4: Exposing Hidden Crashes with Hypothesis (SS2)

Running `python -m pytest test_car_insurance_premium_parser.py` immediately falsified the functions' unhandled assumptions. Hypothesis uncovered three distinct failure classes and shrunk them to minimal counterexamples:

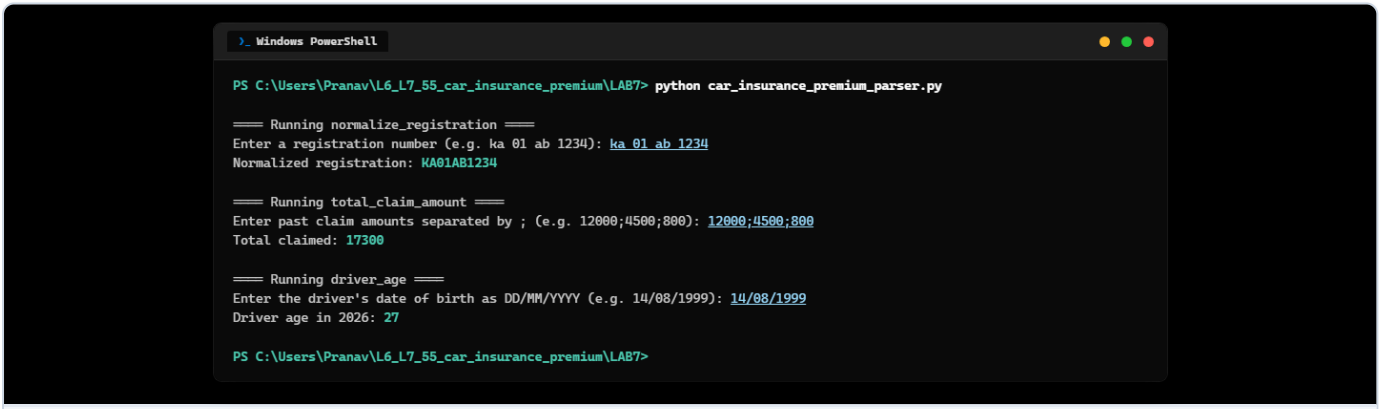
- **`normalize_registration(plate)`:** Crashed with `AttributeError: 'NoneType' object has no attribute 'strip'` when `s=None`.
- **`total_claim_amount(text)`:** Crashed with `AttributeError` on `s=None` and `ValueError: invalid literal for int() with base 10: ''` on empty string `s=''` and malformed parts.
- **`driver_age(dob)`:** Crashed with `AttributeError` on `s=None`, `IndexError: list index out of range` on `s=''` (insufficient slash partitions), and `ValueError` on non-integer year tokens.



Screenshot 2 (SS2): Output of `pytest test_car_insurance_premium_parser.py` showing Hypothesis exposing failures across all 3 functions. 3 FAILED

5. Step 5: Hardening the Application & Re-Execution (SS3)

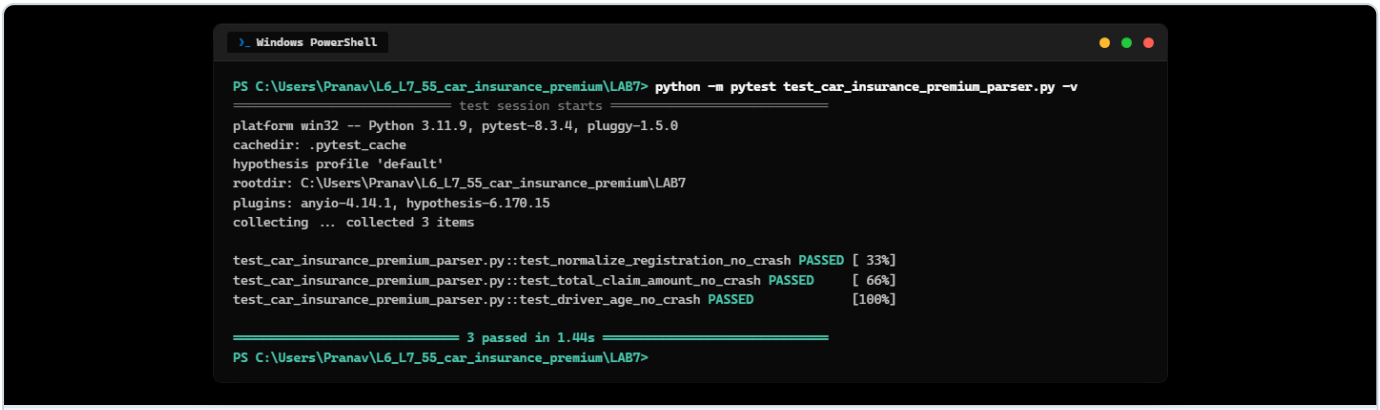
The application was hardened with defensive checks: type inspection (`isinstance(..., str)`), whitespace stripping, token filtering, try/except conversion guards, list length checks (`len(parts) >= 3`), and safe fallback values.



Screenshot 3 (SS3): Execution of the hardened `car_insurance_premium_parser.py` confirming backward compatibility with valid inputs.

6. Step 6: Regression Fuzzing on the Fixed Module (SS4)

Re-running the Hypothesis fuzz suite against the hardened module shows 100% test passage. Hypothesis generated hundreds of malformed inputs across all test functions without triggering any uncaught exceptions.



Screenshot 4 (SS4): Output of `pytest test_car_insurance_premium_parser.py -v` confirming all Hypothesis fuzz tests pass. 3 PASSED

7. Reflection and Analysis

Question 1: What kinds of bugs or edge cases did Hypothesis uncover that normal unit testing might miss?

Traditional unit testing focuses primarily on developer-chosen happy paths and common negative examples. Hypothesis systematically generated extreme inputs that exposed implicit assumptions:

- **Null Pointer Analogues:** None immediately broke unchecked string method calls (`.strip()`, `.split()`), yielding `AttributeError`.
- **Empty Tokens:** Empty strings `""` or consecutive semicolons `";;"` caused `int("")` to throw `ValueError`.
- **Partition Underflows:** Arbitrary strings missing slash separators caused `IndexError` when accessing `parts[2]` in `driver_age`.

Question 2: How does property-based testing differ from traditional example-based unit testing?

Example-based unit testing tests specific concrete input-output values (e.g. `driver_age("14/08/1999") == 27`). While useful, it only tests what the developer thinks to test.

Property-based testing tests general system *invariants* (such as "the parser must never crash on any input"). Hypothesis automatically generates hundreds of randomized inputs to disprove the property, and when a failure occurs, it applies **shrink algorithms** to present the smallest minimal reproducing example (e.g., shrinking complex strings to `""` or `None`).

Question 3: What defensive programming techniques were used to make the module robust?

Four core defensive programming strategies were applied to secure the parser:

1. **Type Invariant Guards:** Validating `if not isinstance(..., str)` at the start of each function to prevent NoneType crashes.
2. **Structural Boundary Checks:** Verifying partition lengths (`len(parts) >= 3`) before array indexing to prevent out-of-range memory/list errors.
3. **Controlled Exception Containment:** Wrapping type conversions (`int()`) in `try/except (ValueError, TypeError)` blocks, discarding malformed tokens without halting parsing.
4. **Safe Fallbacks:** Returning predictable, deterministic default values (`""` for invalid plates, `0` for invalid claim totals or ages).

8. Summary of Bug Findings and Fixes

Function	Initial Bug / Crash	Failing Input	Defensive Fix Applied
`normalize_registration`	`AttributeError` on non-string	`s=None`	Type guard: `if not isinstance(plate, str): return ""`
`total_claim_amount`	`AttributeError`, `ValueError`	`s=None`, `s=""`	Filter empty parts, wrap `int(amount)` in `try/except`
`driver_age`	`AttributeError`, `IndexError`	`s=None`, `s=""`	Check `len(parts) >= 3`, validate integer year, clamp to non-negative

9. Appendix: Hypothesis Test Code (`test_car_insurance_premium_parser.py`)

```
from hypothesis import given, strategies as st from car_insurance_premium_parser import normalize_registration, total_claim_amount, driver_age @given(st.text() | st.none()) def test_normalize_registration_no_crash(s): try: normalize_registration(s) except Exception as e: assert False, f"normalize_registration crashed with: {e}" @given(st.text() | st.none()) def test_total_claim_amount_no_crash(s): try: total_claim_amount(s) except Exception as e: assert False, f"total_claim_amount crashed with: {e}" @given(st.text() | st.none()) def test_driver_age_no_crash(s): try: driver_age(s) except Exception as e: assert False, f"driver_age crashed with: {e}"
```